

Company: Microsoft

Interview Type: DSA

Q1: Service Dependency Graph: Finding Critical Bottlenecks

Difficulty: Hard

Tags: Graph, Tarjan's Algorithm, Connectivity

Problem Statement: In Microsoft Azure, services are represented as nodes in a Directed Graph, and dependencies are directed edges. A "Critical Bottleneck" is defined as a service which, if it fails (along with its outgoing edges), would increase the number of disconnected components in the underlying undirected representation of the network. Given a list of service connections, identify all Critical Bottleneck services.

Expected Approach: Represent the network as an undirected graph. Use a Depth First Search (DFS) to find articulation points. An articulation point is a vertex that, when removed, increases the number of connected components.

Optimized Approach: Use Tarjan's Algorithm or Hopcroft-Tarjan for finding articulation points. Maintain `discovery\_time` and `low\_link\_value` for each node. During DFS, for an edge (u, v), if `low\_link\_value[v] >= discovery\_time[u]`, then `u` is an articulation point (with special handling for the root of the DFS tree).

Time Complexity: $O(V + E)$ where V is the number of services and E is the number of dependencies.

Space Complexity: $O(V)$ to store the discovery times, low-link values, and the adjacency list.

Optimizations: Use an adjacency list instead of a matrix. Use an iterative DFS if the graph depth exceeds the recursion limit to avoid StackOverflow.

Hints: Think about how the connectivity changes if a node is removed. How can we identify a back-edge in a DFS tree?

Q2: Efficient Log Stream: Most Frequent K-length Sequence

Difficulty: Medium

Tags: String, Rolling Hash, Sliding Window

Problem Statement: A Microsoft Teams log file contains a long string of telemetry events (e.g., "ABCCBABC"). To optimize storage, you need to find the most frequently occurring sequence of events of length exactly `K`. If multiple sequences have the same maximum frequency, return any one of them.

Expected Approach: Use a sliding window of size `K` and a hash map to store the frequency of every substring.

Optimized Approach: Use the Rabin-Karp Rolling Hash technique to calculate the hash of the current window in $O(1)$ time as it slides. Store the hash frequencies in a Hash Map. To handle

collisions, verify the actual string only when hashes match, or use a double-hashing strategy to minimize collision probability.

Time Complexity: $O(N)$ where N is the length of the telemetry string.

Space Complexity: $O(N)$ to store the frequencies of the hashes.

Optimizations: Instead of storing the actual substring as a key in the map, store the 64-bit hash value to save space. Use a sliding window to avoid unnecessary string allocations.

Hints: How can you update the hash of a window when one character leaves and another enters? What is the mathematical formula for a rolling hash?

Q3: Minimum Execution Time for Task Graphs with Resource Constraints

Difficulty: Hard

Tags: Graph, Greedy, Priority Queue, Topological Sort

Problem Statement: You are given N tasks to be executed on a single-core processor. Some tasks have dependencies represented as a Directed Acyclic Graph (DAG), where an edge (u, v) means task u must complete before v starts. Each task i has an execution time T_i and a "Resource ID" R_i . Only one task can run at a time. If the processor switches between tasks with different Resource IDs, a context-switch penalty P is added to the total time. Determine the minimum time required to complete all tasks.

Expected Approach: Use topological sort to track available tasks (those with in-degree 0). A naive approach might pick any available task, but this ignores the context-switch penalty.

Optimized Approach: Maintain a Priority Queue of tasks whose dependencies are met. Group available tasks by their Resource ID. If the current resource being processed has more tasks available in the "ready" pool, prioritize them to avoid the penalty P . If no tasks of the current Resource ID are available, pick the Resource ID that unlocks the most immediate "bottleneck" tasks or use a look-ahead greedy strategy to minimize total switches. For a single-core constraint with fixed P , the problem reduces to minimizing the number of switches.

Time Complexity: $O((N + E) \log N)$ where N is tasks and E is dependencies.

Space Complexity: $O(N + E)$ to store the graph and in-degree counts.

Optimizations: Use a frequency map within the ready-set to instantly identify if the "current" resource is available.

Hints: Think about this as a variation of Kahn's algorithm. How does the penalty P change your selection criteria from the "ready" queue?

Q4: Longest Consistent Sub-sequence in Distributed Version Logs

Difficulty: Medium-Hard

Tags: Dynamic Programming, Sliding Window, Hash Map

Problem Statement: A distributed system generates logs across M nodes. Each log entry is a string ID. Due to network jitter, nodes might record events in slightly different orders. You are given M arrays representing these log sequences. Find the length of the longest "consistent" sub-sequence that appears in at least K out of M nodes. A sub-sequence is consistent if the elements appear in the same relative order and the gap between any two consecutive elements in the sub-sequence does not exceed a distance D in the original arrays.

Expected Approach: Standard Longest Common Subsequence (LCS) logic, but restricted by the distance D and scaled to K arrays.

Optimized Approach: Use a sliding window combined with a frequency map or a DP table with a pruning condition. For each element, store its indices across all arrays in a hash map. Iterate through the first array and use DP where $DP[i][j]$ represents the longest sequence ending at index i in array 1 and index j in array n . Incorporate the distance constraint D by only checking the previous D indices in the DP state or using a Fenwick tree/Segment tree to optimize the range maximum query.

Time Complexity: $O(M \cdot N \cdot D)$ or $O(M \cdot N \cdot \log N)$ depending on the implementation.

Space Complexity: $O(M \cdot N)$ to store indices and DP states.

Optimizations: Use rolling hashes if IDs are long strings to reduce comparison time to $O(1)$.

Hints: If D is small, the search space for the previous element in the sequence is limited. How can you use this to prune the DP?

Q5: Grid Pathfinding with Limited Obstacle Removal

Difficulty: Hard

Tags: Graph, Breadth-First Search (BFS), Matrix

Problem Statement: You are given an $m \times n$ grid where each cell is either 0 (empty) or 1 (obstacle). You can move up, down, left, or right. You are allowed to remove at most k obstacles. Find the shortest path from the top-left corner $(0, 0)$ to the bottom-right corner $(m-1, n-1)$. If no such path exists, return -1.

Expected Approach: Use a Breadth-First Search (BFS) to find the shortest path. To handle the obstacle removal, the state must include the current row, column, and the number of remaining obstacle removals available.

Optimized Approach: Implement BFS with a 3D visited array $visited[row][col][remaining_k]$ to track the minimum obstacles removed to reach each cell. This prevents redundant processing of states that have fewer "removals" left for the same coordinate.

Time Complexity: $O(m \cdot n \cdot k)$

Space Complexity: $O(m \cdot n \cdot k)$

Optimizations: If $k \geq m + n - 2$, return the Manhattan distance $m + n - 2$ immediately, as you can clear any path.

Hints: Think of the "remaining k " as a third dimension in your graph coordinates.

Q6: Minimum Nodes to Monitor All Edges (Binary Tree Camera)

Difficulty: Hard

Tags: Dynamic Programming, Greedy, Tree, DFS

Problem Statement: Given the root of a binary tree, we install cameras on the nodes of the tree. Each camera at a node can monitor its parent, itself, and its immediate children. Determine the minimum number of cameras needed to monitor all nodes in the tree.

Expected Approach: Use a depth-first search (DFS) with a post-order traversal. For each node, return its state: 0 if the node is not covered, 1 if the node has a camera, or 2 if the node is covered but has no camera.

Optimized Approach: Apply a greedy logic during the post-order DFS. If a child is not covered, the current parent must have a camera. If at least one child has a camera, the current parent is covered. Otherwise, the current node is not covered and needs its parent to have a camera.

Time Complexity: $O(N)$, where N is the number of nodes in the tree.

Space Complexity: $O(H)$, where H is the height of the tree (recursion stack).

Optimizations: Handle the root case specifically: if the root returns a "not covered" state after the recursion, increment the camera count by one.

Hints: It is always more efficient to place cameras on parents of leaf nodes rather than the leaf nodes themselves.

Q7: Hierarchical Resource Locking Manager

Difficulty: Hard

Tags: Tree, Hash Map, Design, Concurrency

Problem Statement: Design an N-ary tree data structure where each node can be locked by a specific user. A node can only be locked if: 1. It is not already locked. 2. None of its ancestors are locked. 3. None of its descendants are locked. Implement three functions: `lock(nodeId, userId)`, `unlock(nodeId, userId)`, and `upgrade(nodeId, userId)`. The `upgrade` function locks the node if the user can unlock all locked descendants and the node itself meets the locking criteria.

Expected Approach: Use a standard tree representation where each node stores its lock state and the ID of the user who locked it. To check ancestors, traverse up to the root. To check descendants, use a recursive search or maintain a "locked_descendant_count" variable in each node.

Optimized Approach: Maintain a `lock_count` and `locked_descendant_count` for each node. When locking a node, check the `lock_count` of all ancestors by traversing up to the root. To check descendants, simply check if the current node's `locked_descendant_count` is 0. Update the `locked_descendant_count` of all ancestors when a lock/unlock is successful. For `upgrade`, ensure the node has at least one locked descendant, all locked descendants are owned by the same user, and no ancestors are locked.

Time Complexity: Lock/Unlock: $O(H)$ where H is tree height. Upgrade: $O(H + \text{Number of descendants})$.

Space Complexity: $O(N)$ to store the tree and metadata.

Optimizations: Use a Map to store node references for $O(1)$ access to node objects by ID.

Hints: Think about how to avoid traversing the entire subtree to check if any descendant is locked.

Q8: Minimum Energy Path with Limited Battery and Recharge Stations

Difficulty: Hard

Tags: Graph, Dijkstra, Shortest Path, State-Space Search

Problem Statement: You are given a directed graph representing a city's power grid. Each edge has a weight representing the energy consumed to travel between nodes. You have an electric vehicle with a maximum battery capacity C . Some nodes are "Charging Stations" where you can refill the battery to C for a fixed cost. Find the path from source S to destination D that minimizes total time (edge weights), provided the battery never drops below zero.

Expected Approach: Use Dijkstra's algorithm. However, the standard Dijkstra only tracks the minimum cost to reach a node. Here, the state must include the remaining battery.

Optimized Approach: Use a modified Dijkstra where the state is a pair $(current_node, current_battery)$. The priority queue will store $(total_time, node, battery)$. For each edge (u, v) with weight w , if $battery \geq w$, you can move to v with $battery - w$. If node u is a charging station, also add an edge representing "charging" which resets battery to C and adds the station's cost to the total time.

Time Complexity: $O(E * C * \log(V * C))$, where V is nodes, E is edges, and C is battery capacity.

Space Complexity: $O(V * C)$ to store the distance array for each state.

Optimizations: If multiple states reach the same node, a state with more battery and less time is always better. Use a 2D array $dist[node][battery]$ to prune redundant paths.

Hints: Treat the battery level as a second dimension of your graph. How does the state change when you cross an edge versus when you charge?

Q9: Minimum Cost to Connect Data Centers with Potential Hubs

Difficulty: Hard

Tags: Graph, Minimum Spanning Tree, Greedy

Problem Statement: You are designing a network for N data centers. You are given an array P where $P[i]$ is the cost to build a direct satellite station at center i . You are also given a list of potential fiber-optic connections between centers i and j with a cost C . Any two centers that both have satellite stations can communicate with each other at no additional cost. Find the minimum total cost to make all N centers interconnected (directly or indirectly).

Expected Approach: Attempting to find the Minimum Spanning Tree (MST) of the fiber connections and then adding the cheapest satellite station does not account for multiple satellite clusters or the possibility that satellite links are cheaper than fiber.

Optimized Approach: Introduce a "Virtual Hub" (Node 0). For every data center i from 1 to N , create an edge between Node 0 and Node i with a weight equal to $P[i]$. Treat these as additional

edges in your graph. Run a standard MST algorithm (like Kruskal's using Disjoint Set Union) on the $N+1$ nodes. The algorithm will automatically decide whether it is cheaper to connect a center via fiber to its neighbors or via a satellite station to the "Virtual Hub."

Time Complexity: $O(E \log E)$ where E is the number of fiber connections plus N virtual edges.

Space Complexity: $O(V + E)$ to store the graph structure and DSU.

Optimizations: Use path compression and union by rank in DSU to keep the complexity nearly linear $O(E \cdot \alpha(V))$.

Hints: How can you represent the "satellite" option as a standard edge? Think about a dummy node that acts as the satellite network itself.

Q10: Maximum Revenue from Time-Constrained Routes with Turbo Boosts

Difficulty: Hard

Tags: Dynamic Programming, Directed Acyclic Graph (DAG), Shortest Path

Problem Statement: A delivery drone must travel through N cities connected by M one-way routes. Each route i to j has a travel time T_{ij} and a revenue R_{ij} earned upon traversal. The drone has a maximum total travel time $T_{\max}$. You are granted K "Turbo Boosts," each allowing you to traverse any single route in 0 time while still collecting the full revenue. Given the drone starts at City 0, find the maximum revenue collectable within $T_{\max}$.

Expected Approach: Standard DP on a DAG where $DP[u][t]$ is the max revenue at city u at time t . This fails to account for the K boosts.

Optimized Approach: Use multi-dimensional DP: $DP[u][t][k]$ represents the maximum revenue at city u , having spent t time and k boosts. For each route u to v with time w and revenue r , the transitions are: 1) $DP[v][t+w][k] = \max(DP[v][t+w][k], DP[u][t][k] + r)$ if $t+w \leq T_{\max}$. 2) $DP[v][t][k+1] = \max(DP[v][t][k+1], DP[u][t][k] + r)$ if $k < K$.

Time Complexity: $O((N + M) \cdot T_{\max} \cdot K)$.

Space Complexity: $O(N \cdot T_{\max} \cdot K)$.

Optimizations: If the graph is a DAG, process nodes in topological order. Space can be reduced to $O(N \cdot K)$ if we only care about the previous time step t .

Hints: Treat the boosts as a way to transition between "layers" of the graph. Each layer represents the number of boosts used.
